Dockerfile – Hands-On Tutorial (Automation-First Approach)

1. Why exporting changes made inside a container is NOT useful for automation

When you install software and create files **manually inside a running container** and then use `docker commit`, you get a working image — but it is **not suitable for automation**.

Problems with container-based changes

1. Not reproducible

- No record of *how* Java was installed
    - No way to rebuild the same image automatically

2. Not version controlled

- Changes cannot be reviewed in Git
    - No history of what changed and why

3. Not deterministic

- Manual steps can differ each time
    - Leads to "works on my machine" issues

4. Not CI/CD friendly

- CI systems cannot replay interactive shell commands
    - Automation requires declarative instructions

5. Poor maintenance

- Updating Java version requires repeating everything manually

- No clear dependency documentation

Conclusion

For **learning and debugging**, `docker commit` is acceptable. For **automation, CI/CD, production, and team usage**, it is the **wrong approach**.

*This is why **Dockerfile exists**.*

2. What is a Dockerfile

A **Dockerfile** is a **text file** containing **step-by-step instructions** to build a Docker image.

Key properties:

- Declarative
- Reproducible
- Version controllable
- Automatable
- Deterministic

3. Dockerfile Keywords (Core Instructions)

FROM

Specifies the base image.

```
FROM ubuntu:22.04
```

`

Rules:

- Must be the first instruction
- Defines OS / runtime environment

4/36

RUN

Executes commands at build time.

```
RUN apt update && apt install -y openjdk-17-jdk
```

Used for:

- Installing packages
- Configuring system

5/36

WORKDIR

Sets working directory inside the image.

```
WORKDIR /home/app
```

- Creates directory if it does not exist
- Replaces `cd`

6/36

COPY

Copies files from host to image.

```
COPY Hello.java .
```

7/36

ADD

Similar to COPY but with extra features.

```
ADD app.tar /home/app
```

Rarely used. Prefer COPY.

8/36

ENV

Sets environment variables.

```
ENV JAVA_HOME=/usr/lib/jvm/java-17-openjdk-amd64
```

9/36

EXPOSE

Documents container listening port.

```
EXPOSE 8080
```

- Does NOT publish the port
- Used for documentation and tooling

10/36

CMD

Default command when container starts.

```
CMD ["java", "Hello"]
```

- Can be overridden at runtime

11/36

ENTRYPOINT

Defines fixed executable.

```
ENTRYPOINT ["java"]
```

Often combined with CMD.

12/36

4. Hands-On: Build Java App Using Dockerfile

Step 1: Project Structure (Host Machine)

```
java-docker/
├── Dockerfile
└── Hello.java
```

13/36

Step 2: Java Program (Hello.java)

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello from Dockerfile automation");
    }
}
```

14/36

Step 3: Dockerfile (Automation-Friendly)

```
FROM ubuntu:22.04

RUN apt update && apt install -y openjdk-17-jdk

WORKDIR /home/app

COPY Hello.java .

RUN javac Hello.java
```

```
CMD ["java", "Hello"]
```

15/36

5. Build Image from Dockerfile

From the project directory:

```
docker build -t java-app:1.0 .
```

Explanation:

- `-t` → image name and tag
- `.` → build context (current directory)
- `-f` → specify alternative dockerfile like `test.Dockerfile`

Verify:

```
docker images
```

16/36

6. Run Container from Built Image

```
docker run java-app:1.0
```

Output:

```
Hello from Dockerfile automation
```

17/36

7. Override CMD at Runtime

```
docker run -it java-app:1.0 bash
```

CMD replaced by `bash`.

18/36

8. Layered Build Concept (Important)

Each Dockerfile instruction creates a **layer**:

```
FROM ubuntu
RUN install java
WORKDIR /home/app
COPY Hello.java
RUN javac Hello.java
CMD java Hello
```

Benefits:

- Faster rebuilds (layer caching)
- Smaller downloads
- Better debugging

19/36

9. Rebuild After Code Change

Modify `Hello.java`, then:

```
docker build -t java-app:1.1 .
```

Only layers after `COPY` are rebuilt.

20/36

10. Share Dockerfile-Based Image

Push to registry

```
docker tag java-app:1.0 username/java-app:1.0
docker push username/java-app:1.0
```

Pull anywhere

```
docker pull username/java-app:1.0
docker run username/java-app:1.0
```


11. Dockerfile vs docker commit (Exam Ready)

Aspect	Dockerfile	docker commit
Automation	Yes	No
Version control	Yes	No
CI/CD	Yes	No
Reproducible	Yes	No
Production use	Yes	No

12. Key Takeaway

If it cannot be rebuilt automatically, it does not scale.

Dockerfile = Infrastructure as Code

CMD and ENTRYPOINT in Docker, with examples and when to use which.

1. Purpose at a glance

Aspect	CMD	ENTRYPOINT

Main role	Provide **default arguments/command**	Define the **main executable**
Can be overridden at `docker run`	Yes, completely	Not easily (unless `--entrypoint`)
Typical use	Default behavior that user may change	Fixed command that always runs
Flexibility	High	Controlled / strict

2. CMD

What it does

- Specifies the **default command or arguments** for a container.
- If the user passes a command during `docker run`, **CMD is replaced**.

Example

```
FROM ubuntu
CMD ["echo", "Hello World"]
```

Run without override

```
docker run myimage
```

Output:

```
Hello World
```

Run with override

```
docker run myimage echo "Hi"
```

Output:

```
Hi
```

Key point: CMD is **easy to override**.

26/36

3. ENTRYPOINT

What it does

- Defines the **main process** that will always run.
- Arguments passed in `docker run` are **appended** to ENTRYPOINT (not replaced).

Example

```
FROM ubuntu
ENTRYPOINT ["echo"]
```

Run

```
docker run myimage Hello
```

Output:

```
Hello
```

Key point: ENTRYPOINT is **not replaced**, it is **extended**.

27/36

4. ENTRYPOINT + CMD together (most important pattern)

This is the **recommended best practice**.

Example

```
FROM ubuntu
ENTRYPOINT ["echo"]
CMD ["Hello World"]
```

Run without arguments

```
docker run myimage
```

Output:

```
Hello World
```

Run with arguments

```
docker run myimage Hi Prateek
```

Output:

```
Hi Prateek
```

How it works internally

```
ENTRYPOINT + CMD → final command
```

If user provides arguments:

```
ENTRYPOINT + docker run args
```


5. Overriding behavior summary

Scenario	CMD	ENTRYPOINT
`docker run image`	Used	Used
`docker run image ls`	Replaced	Appended
`docker run --entrypoint bash image`	Ignored	Replaced

6. Exec form vs Shell form (important)

Exec form (recommended)

```
CMD ["nginx", "-g", "daemon off;"]
```

- Proper signal handling
- PID 1 works correctly

Shell form (not recommended)

```
CMD nginx -g "daemon off;"
```

- Runs inside `/bin/sh`
- Signal handling issues

Same applies to ENTRYPOINT.

30/36

7. When to use what

Use CMD when

- You want a **default command**
- Users should be able to easily override behavior
- Example: development tools, test images

Use ENTRYPOINT when

- The container is meant to behave like a **binary**
- You want controlled execution
- Example: CLI tools, agents, runners

Use both when

- You want a **fixed executable** + **configurable defaults**
- Example: web servers, databases

31/36

8. Real-world examples

Nginx

```
ENTRYPOINT ["nginx"]
CMD ["-g", "daemon off;"]
```

CLI-style container

```
ENTRYPOINT ["kubectl"]
CMD ["version"]
```

32/36

One-line mental model

- **CMD** → *default arguments*
- **ENTRYPOINT** → *main executable*

33/36

Class code correction

```
ENTRYPOINT ["echo", "Prateek"]
CMD ["java", "Hello"]
```

Docker combines them like this at runtime:

```
echo Prateek java Hello
```

34/36

Correct mental model (very important)

- **ENTRYPOINT** → *the executable*
- **CMD** → *default arguments to that executable*

*You **cannot** use `ENTRYPOINT` to run one command and `CMD` to run another.*

*Docker **does not execute commands sequentially**.*

shell wrapper:

```
ENTRYPOINT ["sh", "-c", "echo Prateek && java Hello"]
```

This works because `sh -c` executes a command string.

35/36

Best Practice Recommendation

For Java containers:

```
ENTRYPOINT ["java", "Hello"]
```

or

```
CMD ["java", "Hello"]
```

No shell, no echo, one main process.

36/36

Quick rule to remember

```
ENTRYPOINT = WHAT runs
CMD        = WITH WHAT arguments
```